

Système d'inventaire Web + Mobile

ASP.NET Core (Clean Architecture) + React + .NET MAUI

Plan détaillé étape par étape — Julien

1. Recommandation : quelle stack pour le front-end Web ?

Recommandation : React + TypeScript, en application décorrélée (SPA) consommant l'API ASP.NET Core.

C'est le choix qui répond le mieux à un besoin professionnel réel et qui montre le plus large éventail de compétences, pour trois raisons :

- Séparation réelle des responsabilités : en découplant totalement le frontend de l'API, tu es forcé de concevoir une vraie API REST autonome — la même que consommera l'app mobile MAUI. C'est exactement la preuve d'architecture « API-first » que les recruteurs cherchent dans ce type de portfolio.
- Marché de l'emploi au Québec : React domine largement les offres frontend (startups, scale-ups, agences), tout en restant très présent en grande entreprise. Angular reste fort dans certains contrats gouvernementaux/bancaires, mais React a un bassin d'emplois plus large et une courbe d'apprentissage plus rapide à valoriser dans un portfolio.
- Diversité des compétences démontrées : Blazor est séduisant car 100% C#, mais il ne montre pas ta maîtrise de l'écosystème JS/TS (le plus demandé). Une stack React + API .NET + Mobile .NET MAUI prouve que tu es à l'aise autant en C# qu'en TypeScript — un profil full-stack beaucoup plus vendeur.

Stack recommandée : React 18 + TypeScript + Vite, TailwindCSS pour le style, TanStack Query pour la gestion des appels API/cache, React Router, React Hook Form + Zod pour les formulaires, Recharts pour les graphiques du dashboard.

Si tu préfères rester 100% dans l'écosystème .NET (par exemple pour un contrat où c'est explicitement demandé), Blazor WebAssembly reste une alternative honnête. Mais pour un portfolio destiné à maximiser les opportunités, React + TypeScript est le choix le plus stratégique.

2. Vue d'ensemble du projet et stack technique retenue

Stack complète retenue pour la suite du plan :

Composant	Technologie retenue
API Backend	ASP.NET Core 8 Web API
Base de données	PostgreSQL + EF Core
Frontend Web	React 18 + TypeScript + Vite + Tailwind + TanStack Query
App Mobile	.NET MAUI + MVVM Community Toolkit + SQLite
Scan codes	Zxing.Net.MAUI (QR / Barcode)
Authentification	ASP.NET Core Identity + JWT + Refresh Tokens
Temps réel	SignalR (remplace avantageusement les WebSockets bruts)
Validation	FluentValidation
Mapping	Mapster (ou AutoMapper)

Composant	Technologie retenue
Médiateur CQRS	MediatR
Export	ClosedXML (Excel) / CsvHelper (CSV)
Logging	Serilog
Tests backend	xUnit + Moq + FluentAssertions + Testcontainers
Tests frontend	Vitest + React Testing Library
Documentation API	Swagger / OpenAPI
Conteneurisation	Docker + docker-compose
CI/CD	GitHub Actions
Déploiement	Render/Railway/Azure (API) + Vercel (React)

3. Architecture Clean Architecture — Backend ASP.NET Core

La solution .NET est découpée en 4 projets, avec une règle stricte : les dépendances ne pointent que vers l'intérieur (API → Infrastructure → Application → Domain). Le Domain ne dépend de rien.

3.1 Domain (cœur métier, zéro dépendance externe)

- Entités : Product, Stock, Warehouse, Supplier, StockMovement, ApplicationUser, Role
- Value Objects : SKU, Quantity, Money (évite les primitive obsessions)
- Domain Events : StockLowEvent, StockDepletedEvent, MovementRecordedEvent
- Exceptions métier : InsufficientStockException, InvalidMovementException
- Interfaces de règles métier pures (aucune dépendance à EF Core, ASP.NET, etc.)

3.2 Application (cas d'usage, orchestration)

- Pattern CQRS avec MediatR : un Command ou une Query par cas d'usage (CreateProductCommand, GetLowStockQuery, RecordMovementCommand...)
- DTOs d'entrée/sortie, jamais d'exposition directe des entités Domain
- Interfaces des repositories et services externes (IProductRepository, IStockRepository, IExportService, IEmailService) — implémentées plus tard dans Infrastructure (principe DIP)
- Validation des commandes avec FluentValidation, branchée via un Behavior MediatR (pipeline)
- Mapping Entités ↔ DTOs avec Mapster

3.3 Infrastructure (détails techniques)

- DbContext EF Core + configurations d'entités (Fluent API, pas de Data Annotations pour rester découplé)
- Implémentation des repositories + pattern Unit of Work

- Migrations EF Core vers PostgreSQL
- Implémentation des services externes : export Excel/CSV, envoi d'emails/notifications, hub SignalR

3.4 API (présentation, composition root)

- Controllers minces : reçoivent la requête, envoient une Command/Query via MediatR, renvoient le résultat — aucune logique métier ici
- Middlewares : gestion centralisée des exceptions (ProblemDetails), logging des requêtes, rate limiting
- Swagger/OpenAPI avec versioning d'API (api/v1/...)
- Configuration de l'injection de dépendances (composition root) et de l'authentification JWT

4. Principes SOLID appliqués concrètement

Principe	Application concrète dans le projet
S — Responsabilité unique	Un handler MediatR = un seul cas d'usage. Un repository = un seul agrégat (IProductRepository ≠ IStockRepository).
O — Ouvert/fermé	Stratégie d'export (IExportStrategy) : ajouter un nouveau format (PDF) sans modifier le code existant, juste une nouvelle implémentation.
L — Substitution de Liskov	Toute implémentation de IStockRepository (EF Core, mock en test, InMemory) doit respecter le même contrat sans surprise.
I — Ségrégation des interfaces	Interfaces fines et ciblées (IReadProductRepository / IWriteProductRepository si besoin) plutôt qu'un « god repository » générique.
D — Inversion des dépendances	Application dépend d'abstractions (interfaces) ; Infrastructure les implémente et est injectée via le conteneur DI. Aucune référence d'Application vers Infrastructure.

5. Bonnes pratiques Async/Await

- Async de bout en bout : dès qu'une couche est async (accès DB, HTTP), toute la chaîne d'appel l'est aussi — jamais de mélange sync/async.
- Jamais de .Result ou .Wait() sur une Task (risque de deadlock) ; toujours await.
- Jamais d'async void, sauf handlers d'événements UI — dans MAUI et l'API, toujours async Task.
- Propagation systématique d'un CancellationToken depuis le Controller jusqu'à la requête EF Core.
- Utiliser Task.WhenAll pour paralléliser des appels indépendants (ex. charger produit + fournisseur en même temps).
- ConfigureAwait(false) dans les couches Application/Infrastructure (librairies), pas nécessaire dans les Controllers ASP.NET Core moderne.

- `IAsyncEnumerable<T>` pour le streaming de gros exports (éviter de charger 100k lignes en mémoire d'un coup).
- Aucun appel bloquant (I/O synchrone) dans un constructeur ou une propriété.

6. Sécurité — Authentification et autorisation

- ASP.NET Core Identity pour la gestion des utilisateurs (hash de mot de passe sécurisé, verrouillage après tentatives échouées).
- Login → émission d'un JWT (courte durée de vie, ex. 15 min) + un Refresh Token (longue durée, stocké de façon sécurisée).
- Endpoint `/api/auth/refresh` pour renouveler le token sans reconnexion — utilisé par le mobile pour les sessions longues.
- Rôles : Admin, Gestionnaire, Employé — gérés via des Politiques ASP.NET Core (`[Authorize(Policy = "GestionnaireOuPlus")]`).
- CORS restreint aux origines connues (React en dev/prod).
- Rate limiting sur les endpoints sensibles (login) pour limiter le brute force.
- HTTPS obligatoire partout, secrets (connection strings, clé JWT) via variables d'environnement / User Secrets, jamais en dur dans le code.

7. Architecture Frontend Web (React)

- Organisation par feature (feature-based), pas par type de fichier : `src/features/products`, `src/features/movements`, `src/features/dashboard`...
- TanStack Query pour tout appel API : cache, revalidation automatique, gestion des états loading/error sans boilerplate.
- React Hook Form + Zod pour la validation des formulaires (schémas partagés/alignés avec les DTOs backend).
- React Router avec route guards basés sur le rôle utilisateur (protéger `/admin`, `/dashboard` selon le rôle).
- Recharts pour les KPIs du dashboard (valeur du stock, mouvements par période, ruptures).
- Client Axios ou fetch typé avec intercepteur pour attacher le JWT et gérer le refresh automatique en cas de 401.
- Composants UI réutilisables (design system minimal) + Tailwind pour la cohérence visuelle.
- Tests : Vitest + React Testing Library pour les composants critiques (formulaires, tableau de stock).

8. Architecture Mobile MAUI — Clean Architecture + MVVM

Le mobile reprend la même philosophie Clean Architecture, adaptée au contexte MVVM :

- Domain : entités partagées (idéalement dans une class library commune référencée par l'API et le mobile, pour éviter la duplication des règles métier simples).
- Application : cas d'usage mobile (`ScanProductUseCase`, `RecordMovementUseCase`, `SyncUseCase`), interfaces `IApiClient` / `ILocalRepository`.

- Infrastructure : client API (Refit ou HttpClient typé) consommant exactement les mêmes endpoints que React ; repository SQLite (via sqlite-net-pcl ou EF Core Sqlite provider) ; SyncService.
- Presentation : Views XAML + ViewModels avec le MVVM Community Toolkit ([ObservableProperty], [RelayCommand]) — zéro code-behind métier.
- Injection de dépendances configurée dans MauiProgram.cs, exactement comme la composition root de l'API.

8.1 Scan QR / Codes-barres

- Librairie ZXing.Net.MAUI : scan caméra en temps réel, lecture QR + Code128/EAN13.
- Résultat du scan → recherche locale (SQLite) si offline, sinon requête API → affichage fiche produit.

8.2 Mode offline et stockage local

- Base SQLite locale miroir simplifié du modèle serveur (Products, StockMovements en cache).
- Chaque mouvement créé offline est stocké avec un GUID généré côté client + un flag IsSynced = false (pattern Outbox).
- Un service en arrière-plan détecte la reconnexion réseau (IConnectivity) et déclenche la synchronisation.

8.3 Synchronisation et résolution de conflits

- Le GUID client garantit l'idempotence : renvoyer deux fois le même mouvement ne le duplique pas côté serveur.
- Stratégie de résolution de conflits : Last-Write-Wins basé sur l'horodatage serveur (simple, suffisant pour ce cas d'usage — un mouvement d'inventaire est une action, pas un état partagé complexe).
- Chaque entité synchronisable a un champ UpdatedAtUtc comparé entre version locale et serveur.

8.4 Notifications push

- Firebase Cloud Messaging (via Plugin.Firebase ou .NET MAUI Community Toolkit) pour les alertes de stock bas et anomalies, déclenchées côté API lors d'un domain event StockLowEvent.

9. Communication Web ↔ Mobile

- Un seul contrat d'API REST (Products, Stocks, Movements, Warehouses) consommé de façon identique par React et MAUI — c'est la preuve concrète que l'API est vraiment découplée du frontend.
- SignalR plutôt que des WebSockets bruts : SignalR utilise WebSockets sous le capot avec fallback automatique, gère la reconnexion, et s'intègre nativement à ASP.NET Core avec DI. C'est le choix idiomatique et le plus « pro » en .NET.
- Hub SignalR StockHub : notifie le dashboard React en temps réel quand un employé mobile enregistre un mouvement (mise à jour live des KPIs sans refresh).

10. Tests et qualité

Couche	Type de test	Outils
Domain / Application	Tests unitaires (règles métier, handlers)	xUnit, Moq, FluentAssertions
Infrastructure / API	Tests d'intégration (vraie base PostgreSQL éphémère)	WebApplicationFactory, Testcontainers
Frontend React	Tests de composants	Vitest, React Testing Library
Frontend React	Tests end-to-end (optionnel)	Playwright
Mobile MAUI	Tests unitaires des ViewModels	xUnit, Moq

Objectif réaliste : couvrir en priorité les règles métier (calcul de stock, alertes de seuil) et les cas d'usage critiques (création de mouvement, synchronisation), plutôt que de viser un pourcentage de couverture arbitraire.

11. CI/CD et déploiement

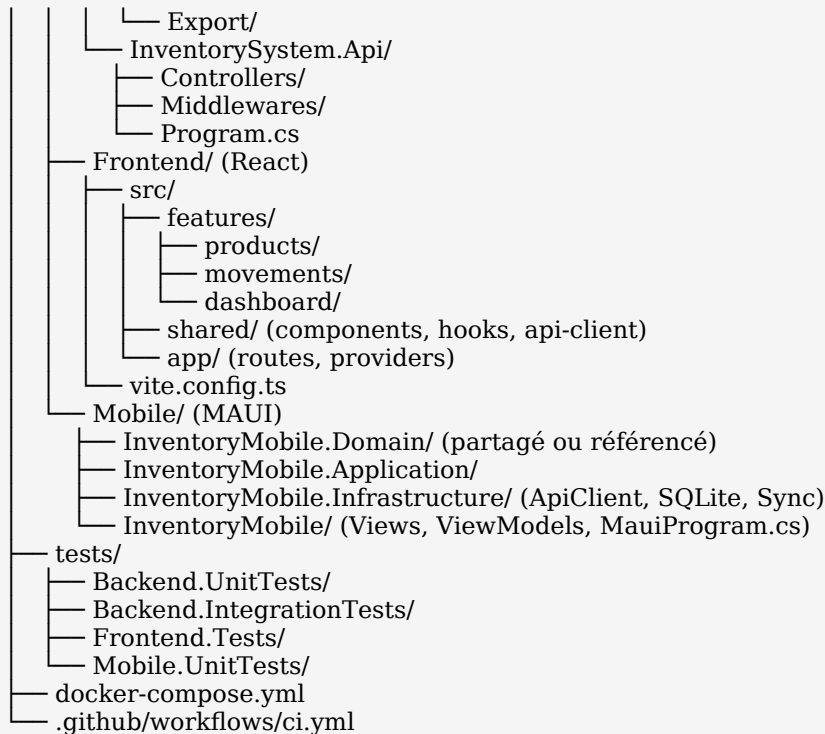
- Docker : Dockerfile pour l'API, docker-compose.yml avec API + PostgreSQL + pgAdmin pour le développement local.
- GitHub Actions : pipeline build → tests unitaires/intégration → lint → build image Docker → déploiement automatique sur push vers main.
- Déploiement API : Render, Railway ou Azure App Service (offre gratuite/étudiante suffisante pour une démo portfolio).
- Déploiement Frontend : Vercel ou Netlify (déploiement automatique depuis GitHub).
- Mobile : build d'un APK de démonstration téléchargeable depuis le README, ou publication interne via TestFlight/Play Console interne si tu veux aller plus loin.

12. Structure de dossiers proposée

```

InventorySystem/
├── src/
│   ├── Backend/
│   │   ├── InventorySystem.Domain/
│   │   │   ├── Entities/
│   │   │   ├── ValueObjects/
│   │   │   ├── Events/
│   │   │   └── Exceptions/
│   │   ├── InventorySystem.Application/
│   │   │   ├── Products/ (Commands, Queries, Validators, DTOs)
│   │   │   ├── Stocks/
│   │   │   ├── Movements/
│   │   │   ├── Common/Interfaces/
│   │   │   └── Common/Behaviors/
│   │   └── InventorySystem.Infrastructure/
│   │       ├── Persistence/ (DbContext, Configurations, Repositories)
│   │       ├── Auth/
│   │       └── SignalR/

```



13. Feuille de route — ordre d'implémentation recommandé

Aucune contrainte de temps n'étant fixée, voici l'ordre logique à suivre, chaque étape s'appuyant sur la précédente :

1. Créer la solution .NET (4 projets Domain/Application/Infrastructure/Api) et le repo Git avec structure Clean Architecture vide.
2. Modéliser le Domain : entités, value objects, règles métier de base (sans EF Core, sans framework).
3. Écrire les premiers tests unitaires du Domain (ex. « un mouvement de sortie ne peut pas rendre le stock négatif ») avant même de brancher la DB.
4. Mettre en place PostgreSQL + EF Core dans Infrastructure, migrations initiales, implémentation des repositories.
5. Implémenter les premiers cas d'usage Application avec MediatR + FluentValidation (CreateProduct, RecordMovement).
6. Exposer les premiers Controllers API + Swagger, tester manuellement avec des requêtes HTTP.
7. Ajouter l'authentification (Identity + JWT + refresh token) et les rôles/policies.
8. Compléter les endpoints restants (Stocks, Warehouses, Suppliers, alertes de stock bas, export Excel/CSV).
9. Écrire les tests d'intégration API (WebApplicationFactory + Testcontainers).
10. Démarrer le frontend React : setup Vite/Tailwind, client API typé, authentification, layout de base.

11. Construire les écrans CRUD (produits, entrepôts, fournisseurs) puis l'historique des mouvements.
12. Construire le dashboard (KPIs, graphiques Recharts) branché sur les endpoints d'agrégation.
13. Ajouter SignalR côté API (StockHub) et l'écoute temps réel côté React.
14. Démarrer le projet MAUI : structure Clean Architecture mobile, connexion à l'API (login JWT).
15. Implémenter le scan QR/Barcode et la création de mouvements en ligne.
16. Ajouter SQLite local, le pattern Outbox et le service de synchronisation offline.
17. Ajouter les notifications push (stock bas) déclenchées par les domain events côté API.
18. Dockeriser, mettre en place la pipeline GitHub Actions, déployer API + Frontend + rédiger le README final avec captures d'écran et schéma d'architecture.

14. Conseils pour un portfolio qui se démarque

- README détaillé : contexte, stack, schéma d'architecture (Clean Architecture + flux Web/Mobile), instructions de lancement local (docker-compose up).
- Un schéma d'architecture visuel (diagramme des couches + flux JWT + flux de synchronisation) — vaut souvent plus qu'un paragraphe.
- Une courte vidéo de démo (2-3 min) : login, scan mobile, mouvement, mise à jour temps réel du dashboard web.
- Un déploiement live accessible (même limité en ressources) plutôt qu'un simple lien vers le code.
- Documenter explicitement les choix techniques et compromis (pourquoi CQRS, pourquoi SignalR plutôt que WebSockets bruts, pourquoi Last-Write-Wins) — cela montre une compréhension réelle, pas juste de l'exécution.